

Experiment No. 1

Aim : Visualize features and implement k-NN for IRIS dataset for different values of k (1, 3, 5, 7) with 5-fold Cross validation using scikit-learn.

Theory:

1. k-Nearest Neighbors (k-NN) Algorithm

The k-Nearest Neighbors (k-NN) algorithm is a straightforward and effective supervised machine learning method used for both classification and regression tasks. Its fundamental principle is to classify a new, unseen data point by associating it with the majority class of its 'k' closest neighbors in the feature space. The "closeness" or proximity between data points is typically measured using a distance metric, with Euclidean distance being one of the most common choices.

2. The IRIS Dataset

This experiment utilizes the classic Iris dataset, a benchmark dataset in the field of machine learning. It consists of 150 samples belonging to three distinct species of the iris flower: Setosa, Versicolor, and Virginica. For each sample, four features are recorded:

1. Sepal Length
2. Sepal Width
3. Petal Length
4. Petal Width

The dataset is well-suited for illustrating classification algorithms because the classes are generally well-separated, especially the Setosa species from the other two. Visualizing the features, for instance through pair plots or scatter plots, is an important initial step to understand the relationships between features and the separability of the classes.

3. 5-Fold Cross-Validation

To obtain a reliable and unbiased estimate of the model's performance, this experiment employs 5-fold cross-validation. This technique is more robust than a simple train-test split. The procedure is as follows:

- The entire dataset of 150 samples is randomly shuffled and partitioned into five equal-sized subsets or "folds" (each containing 30 samples).
- The model is trained using four of the folds (120 samples) as the training data.
- The remaining fold (30 samples) is used as the test data to evaluate the model's accuracy.

Code:

```
!wget https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data -O /content/iris.csv
```

```
import numpy as np

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier

df = pd.read_csv('/content/iris.csv', header=None)
df.columns = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species']
print("Dataset loaded successfully. Columns are:")
print(df.columns)

display(df.head())

print("\nMissing values:")
print(df.isnull().sum())

sns.pairplot(df, hue='species')
plt.suptitle("Pairplot of Iris Dataset", y=1.02)
plt.show()

plt.figure(figsize=(10, 6))
df.drop('species', axis=1).boxplot()
plt.title("Boxplot of Features")
plt.show()

#BOX PLOT
for col in df.columns[:-1]: # Exclude target
    sns.boxplot(x='species', y=col, data=df)
    plt.title(f'Boxplot of {col} by Target')
    plt.show()

plt.figure(figsize=(8, 5))
sns.heatmap(df.drop('species', axis=1).corr(), annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()

X = df.drop('species', axis=1)
```

```

y = df['species']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

k_values = [1, 3, 5, 7]
cv_scores = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X, y, cv=5)
    cv_scores.append(scores.mean())
    print(f"k={k} -> Cross-validation Accuracy: {scores.mean():.4f}")

# Plot accuracy vs k
plt.figure(figsize=(8, 5))
plt.plot(k_values, cv_scores, marker='o')
plt.title('k-NN Accuracy for different k values (5-fold CV)')
plt.xlabel('Number of Neighbors (k)')
plt.ylabel('Cross-validation Accuracy')
plt.grid(True)
plt.show()

# Final model with best k
best_k = k_values[np.argmax(cv_scores)]
print(f"Best k from CV: {best_k}")
knn_best = KNeighborsClassifier(n_neighbors=best_k)
knn_best.fit(X_train, y_train)
y_pred = knn_best.predict(X_test)

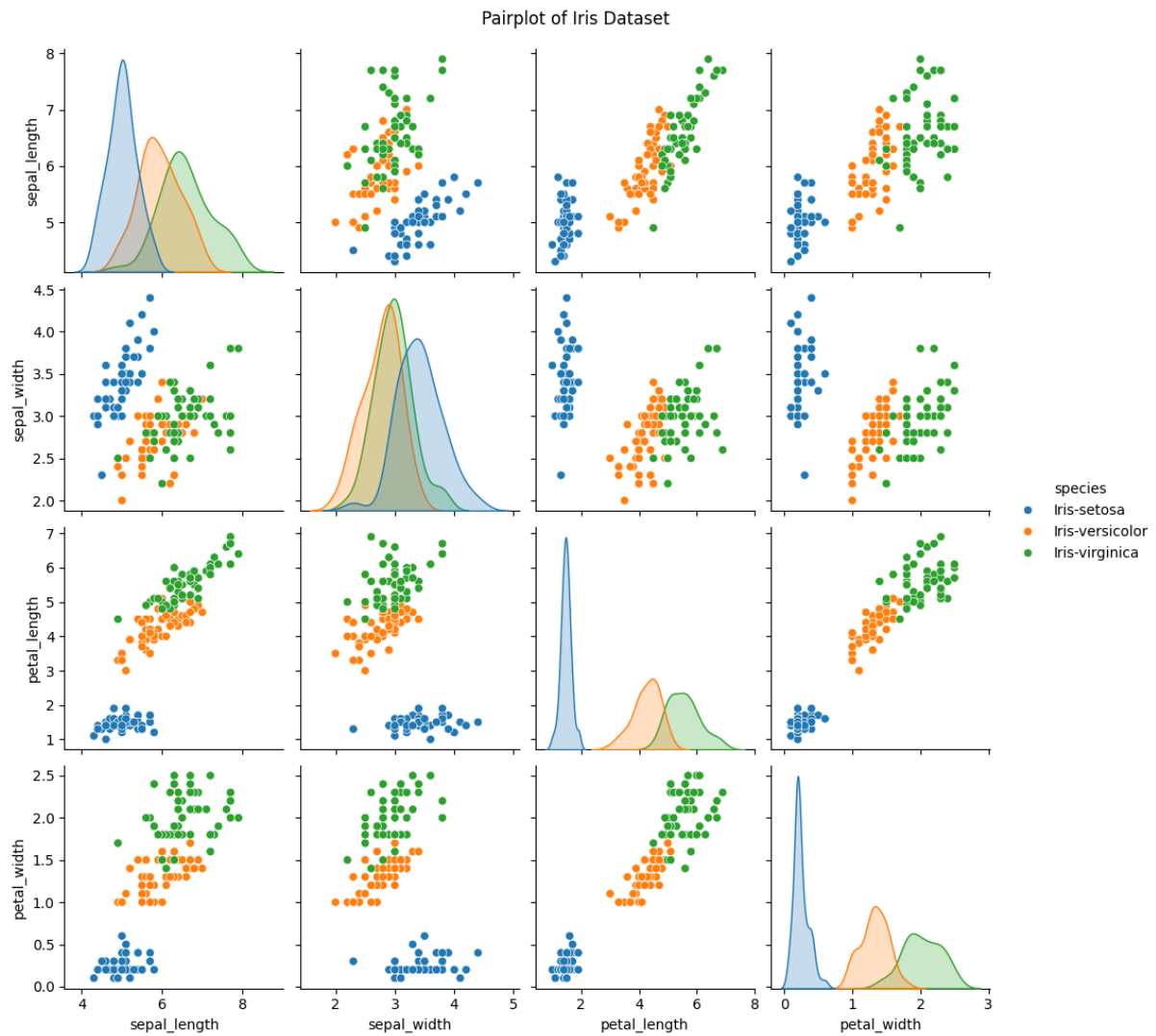
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

print("\nTest Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))

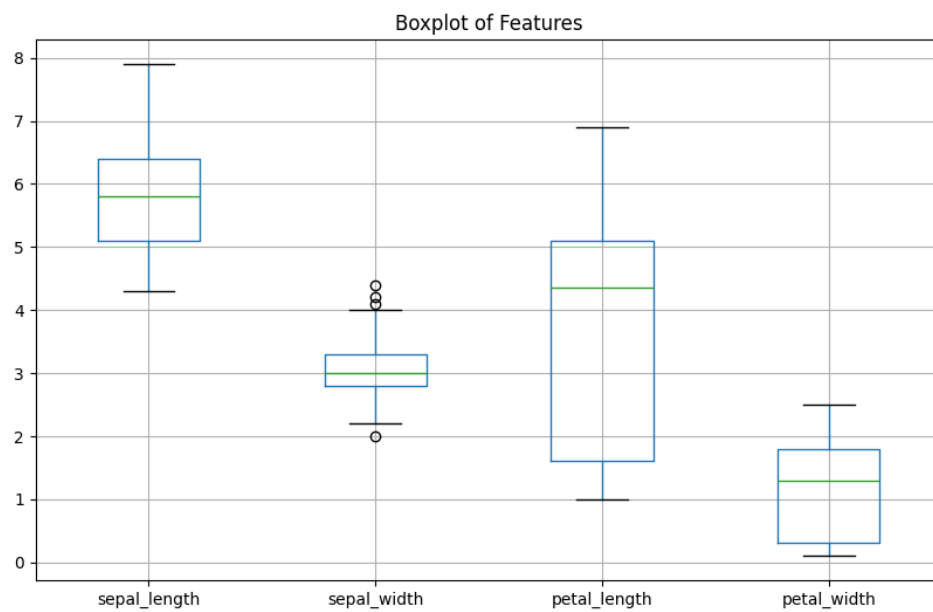
```

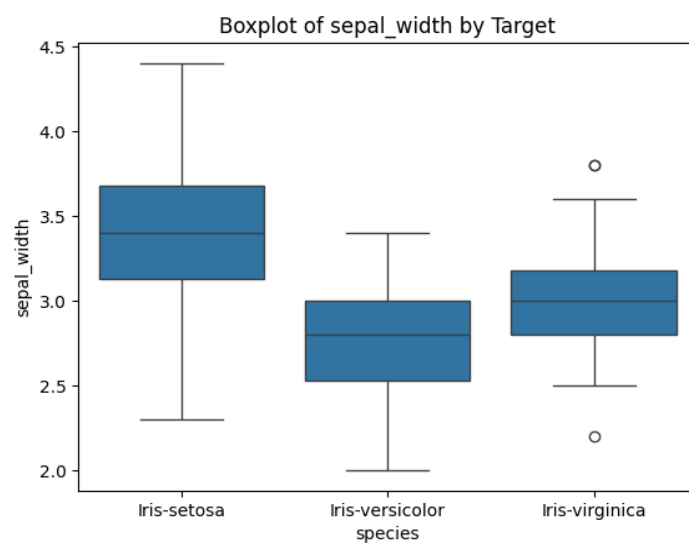
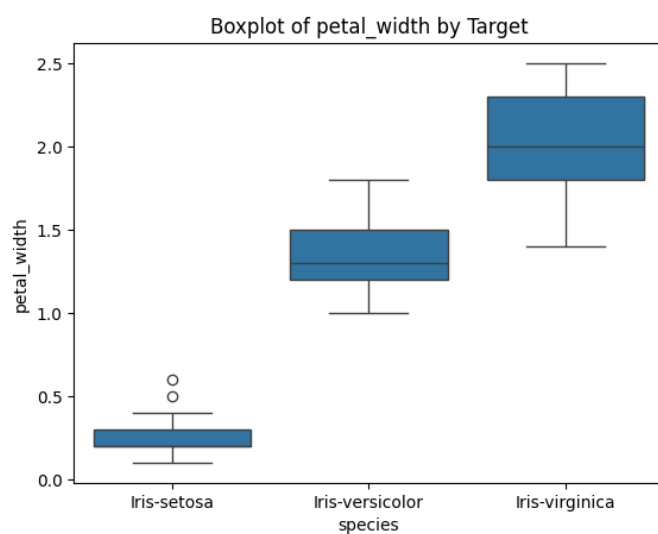
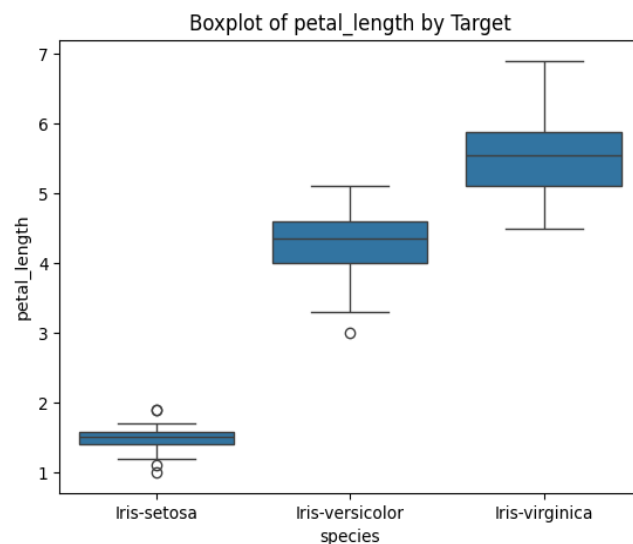
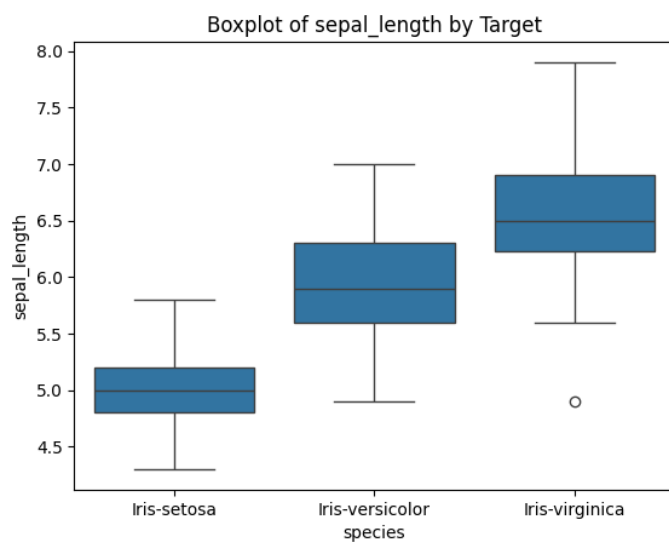
Output:

Pairplot:

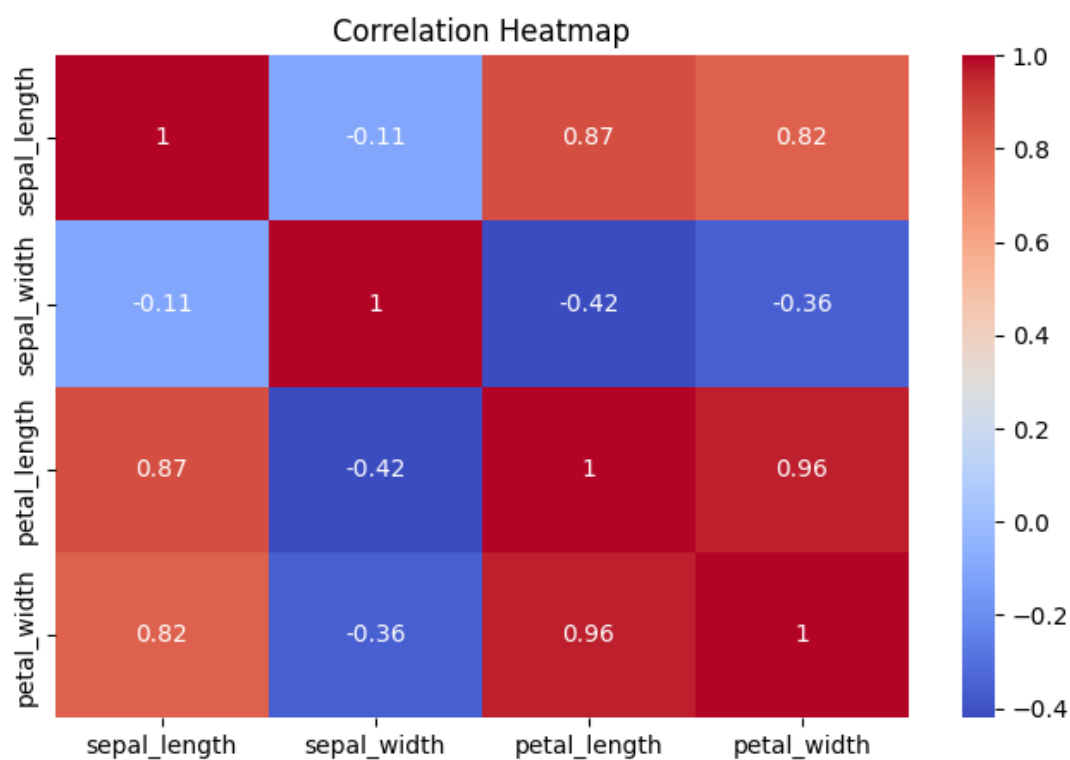


Boxplot:

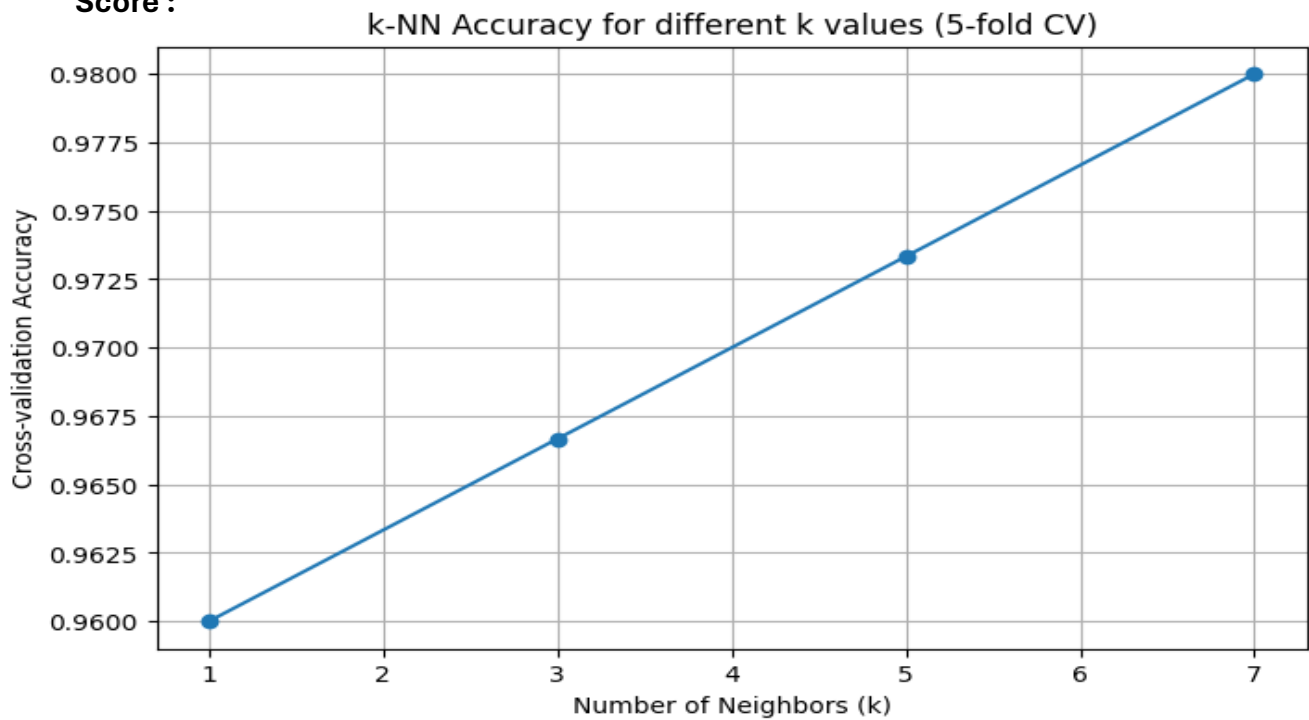




Heatmap:



Score :



```
➤ k=1 -> Cross-validation Accuracy: 0.9600
   k=3 -> Cross-validation Accuracy: 0.9667
   k=5 -> Cross-validation Accuracy: 0.9733
   k=7 -> Cross-validation Accuracy: 0.9800
```

Test Accuracy: 0.9666666666666667

Classification Report:

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	10
Iris-versicolor	0.91	1.00	0.95	10
Iris-virginica	1.00	0.90	0.95	10
accuracy			0.97	30
macro avg	0.97	0.97	0.97	30
weighted avg	0.97	0.97	0.97	30

Confusion Matrix:

```
[[10  0  0]
 [ 0 10  0]
 [ 0  1  9]]
```

Learning Outcomes:

- Load and explore the Iris dataset.
- Visualize features using pairplots, BoxPlot and heatmap.
- Train and test a KNN classifier and understanding of k-fold.
- Connect distance-based learning to vector space.
- Relate classification results to class probabilities.

